

C++ Code

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using int64 = long long;
5
6  int64 gcd(int64 a, int64 b)
7  {
8      while (b != 0)
9      {
10         int64 t = b;
11         b = a % b;
12         a = t;
13     }
14     return a;
15 }
16
17 int64 mul_mod(int64 a, int64 b, int64 mod)
18 {
19     return (int64)((__int128)a * b % mod);
20 }
21
22 int64 mod_pow(int64 base, int64 exp, int64 mod)
23 {
24     int64 result = 1;
25     base %= mod;
26
27     while (exp > 0)
28     {
29         if (exp & 1)
30             result = mul_mod(result, base, mod);
31
32         base = mul_mod(base, base, mod);
33         exp >>= 1;
34     }
35
36     return result;
37 }
38
39 int64 egcd(int64 a, int64 b, int64 &x, int64 &y)
40 {
41     if (b == 0)
42     {
43         x = 1;
44         y = 0;
45         return a;
46     }
47
48     int64 x1, y1;
49     int64 g = egcd(b, a % b, x1, y1);
50
```

```
51     x = y1;
52     y = x1 - (a / b) * y1;
53
54     return g;
55 }
56
57 // Modular Inverse
58 int64 modInverse(int64 e, int64 phi)
59 {
60     int64 x, y;
61     int64 g = egcd(e, phi, x, y);
62
63     if (g != 1)
64         return -1;
65
66     x %= phi;
67     if (x < 0)
68         x += phi;
69
70     return x;
71 }
72
73 int64 autoSelectE(int64 phi)
74 {
75     int64 e = 2;
76
77     while (gcd(e, phi) != 1)
78         e++;
79
80     return e;
81 }
82
83 int main()
84 {
85     int64 p = 10007;
86     int64 q = 10009;
87
88     cout << "Prime p = " << p << endl;
89     cout << "Prime q = " << q << endl;
90
91
92     int64 n = p * q;
93     int64 phi = (p - 1) * (q - 1);
94
95     cout << "n = " << n << endl;
96     cout << "phi = " << phi << endl;
97
98     // Public exponent
99     int64 e = autoSelectE(phi);
100    cout << "Automatically selected e = " << e << endl;
101
102    int64 d = modInverse(e, phi);
103
104    cout << "Public Key (n, e): (" << n << ", " << e << ")" << endl;
```

```
105     cout << "Private Key (d): " << d << endl;
106
107     int64 m1 = 65;
108     int64 m2 = 77;
109
110     int64 c1 = mod_pow(m1, e, n);
111     int64 c2 = mod_pow(m2, e, n);
112
113     cout << "Message 1 = " << m1 << endl;
114     cout << "Message 2 = " << m2 << endl;
115
116     cout << "Encrypted m1 = " << c1 << endl;
117     cout << "Encrypted m2 = " << c2 << endl;
118
119     int64 cprod = mul_mod(c1, c2, n);
120
121     cout << "Combined Ciphertext = " << cprod << endl;
122
123     // Decrypt the product
124     int64 decrypted = mod_pow(cprod, d, n);
125
126     cout << "Decrypted Product = " << decrypted << endl;
127     cout << "Expected Value    = " << (m1 * m2) % n << endl;
128
129     if (decrypted == (m1 * m2) % n)
130         cout << "Equal" << endl;
131     else
132         cout << "not equal" << endl;
133
134     return 0;
135 }
```